Northwind SQL Practice Questions

Parts B — with Solutions and Explanations

Part B — Sorting, Grouping, and Joining

The questions in this section build on Part A by introducing ORDER BY for sorting, GROUP BY and aggregate functions (COUNT, SUM, AVG) for summarization, HAVING for filtering groups, and JOIN for combining data from multiple tables.

Question 1

Task: Write a query to list ProductID, ProductName, UnitPrice, and UnitsInStock from the Products table. Order the results by UnitPrice from highest to lowest.

Hint: Use the ORDER BY clause with DESC to sort in descending order.

Expected Output:

Result Grid | Filter Rows: | Edit: | Export/Import:

ProductID	ProductName	UnitPrice	UnitsInStock
38	Cte de Blaye	263.5000	17
29	Thringer Rostbratwurst	123.7900	0
9	Mishi Kobe Niku	97.0000	29
20	Sir Rodney's Marmalade	81.0000	40
18	Carnarvon Tigers	62.5000	42
59	Raclette Courdavault	55.0000	79
51	Manjimup Dried Apples	53.0000	20
62	Tarte au sucre	49.3000	17
43	Ipoh Coffee	46.0000	17
28	Rssle Sauerkraut	45.6000	26
27	Schoggi Schokolade	43.9000	49
63	Vegie-spread	43.9000	24

Products 1

Output

Action Output

#	Time	Action	Mes
1	08:20:37	USE northwind	0 row
2	08:21:11	SELECT ProductID, ProductName, UnitPrice, UnitsInStock FROM Produ...	77 ro

Question 2

Task: Write a query to list all customers. Order them first by Country (A–Z), then by CompanyName (A–Z) within each country.

Hint: You can list multiple columns in the ORDER BY clause separated by commas.

Expected Output

Result Grid | Filter Rows: | Edit: | Export/Import:

CustomerID	CompanyName	ContactName	Country
VALON	IT	Valon Hoti	NULL
Val2	IT	Val2	NULL
CACTU	Cactus Comidas para llevar	Patricio Simpson	Argentina
OCEAN	Ocano Atlntico Ltda.	Yvonne Moncada	Argentina
RANCH	Rancho grande	Sergio Gutirrez	Argentina
ERNSH	Ernst Handel	Roland Mendel	Austria
PICCO	Piccolo und mehr	Georg Pipps	Austria
MAISD	Maison Dewey	Catherine Dewey	Belgium
SUPRD	Suprmes dlices	Pascale Cartrain	Belgium
COMMI	Comrcio Mineiro	Pedro Afonso	Brazil
FAMIA	Familia Arquibaldo	Aria Cruz	Brazil
GOURL	Gourmet Lanchonetes	Andr Fonseca	Brazil

Customers 2

Output

Action Output

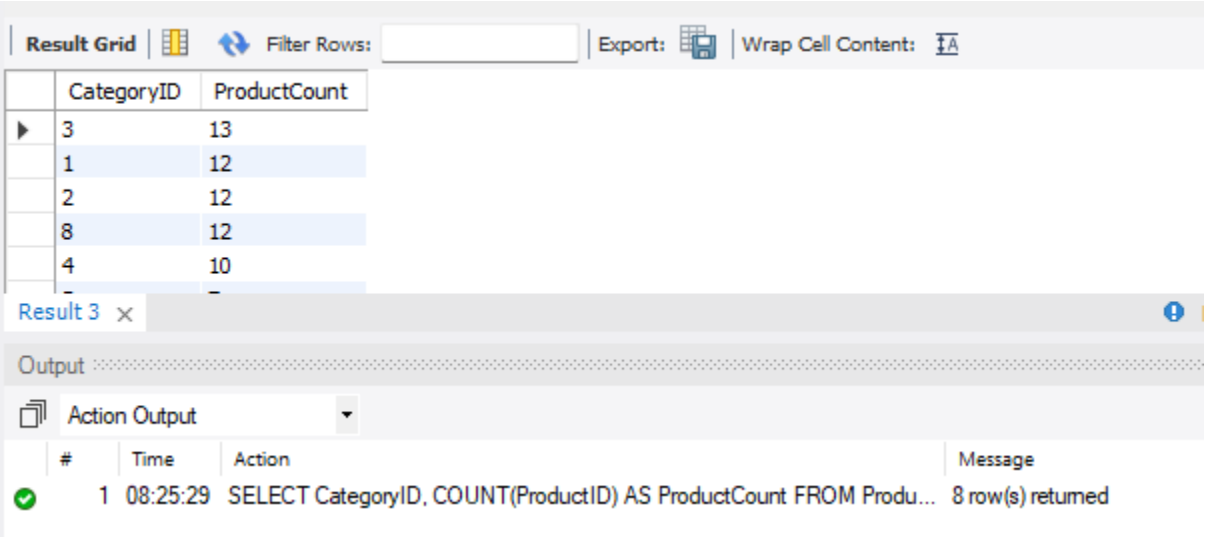
#	Time	Action
1	08:23:14	SELECT CustomerID, CompanyName, ContactName, Country FROM Cus...

Question 3

Task: Write a query that counts the total number of products in each category. Display the CategoryID and the count, labeling the count ProductCount. Order the results so the category with the most products appears first.

Hint: Use COUNT() with GROUP BY on CategoryID, then ORDER BY the count descending.

Expected Output



CategoryID	ProductCount
3	13
1	12
2	12
8	12
4	10

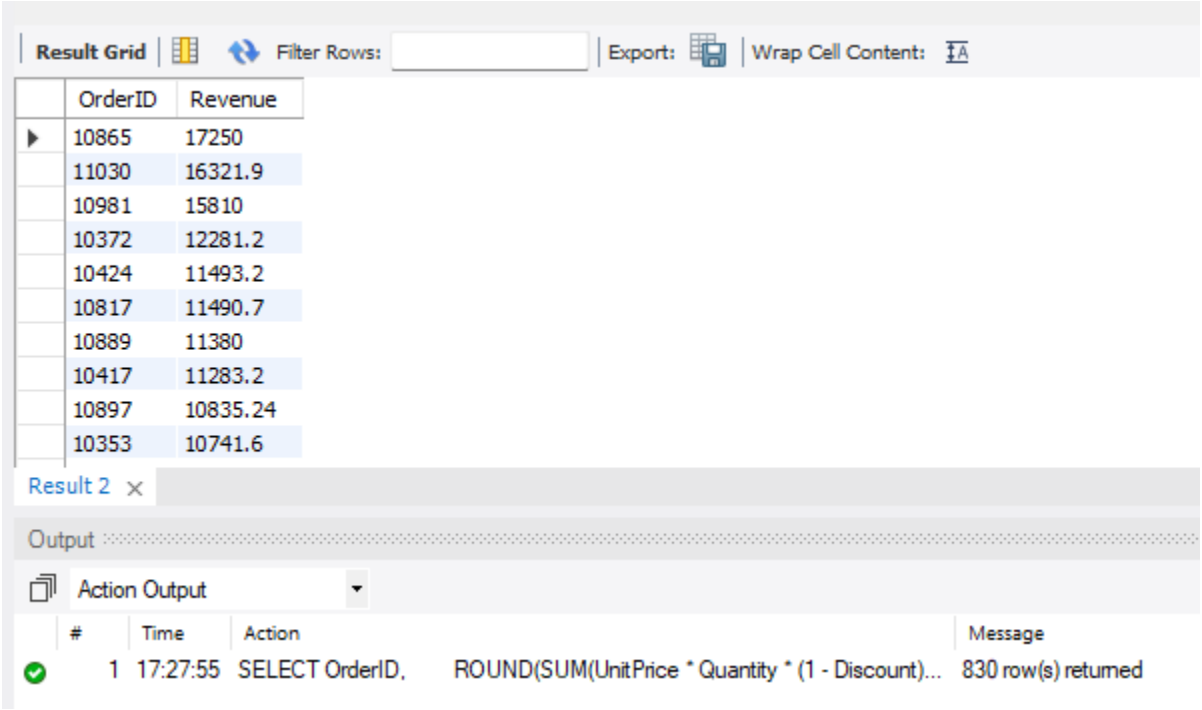
#	Time	Action	Message
1	08:25:29	SELECT CategoryID, COUNT(ProductID) AS ProductCount FROM Produ...	8 row(s) returned

Question 4

Task: Write a query against the Order Details table that calculates the total revenue per order. Total revenue for a line is UnitPrice × Quantity × (1 − Discount). Label the total Revenue and display it alongside OrderID. Order the results by Revenue from largest to smallest.

Hint: Use SUM() on the calculated expression and GROUP BY OrderID.

Expected output



Result Grid | Filter Rows: | Export: | Wrap Cell Content:

OrderID	Revenue
10865	17250
11030	16321.9
10981	15810
10372	12281.2
10424	11493.2
10817	11490.7
10889	11380
10417	11283.2
10897	10835.24
10353	10741.6

Result 2

Output

Action Output

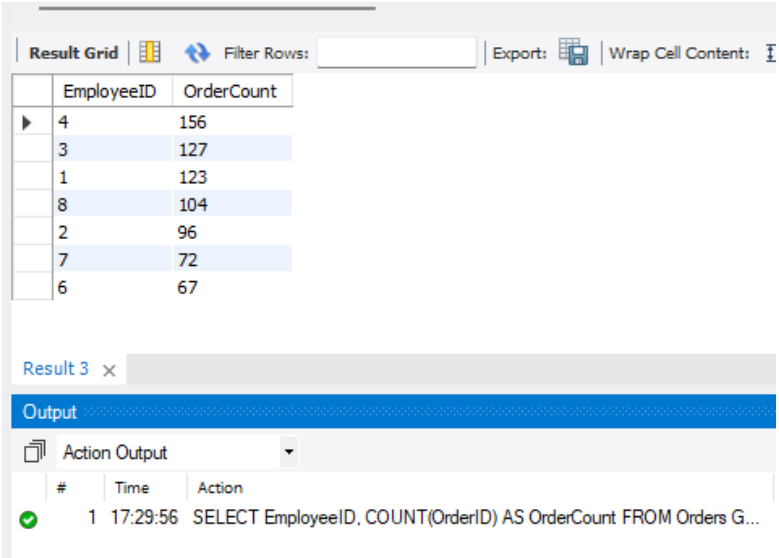
#	Time	Action	Message
1	17:27:55	SELECT OrderID, ROUND(SUM(UnitPrice * Quantity * (1 - Discount)...	830 row(s) returned

Question 5

Task: Write a query to list the number of orders placed by each employee. Show EmployeeID and label the count OrderCount. Only include employees who have handled more than 50 orders. Order by OrderCount, largest first.

Hint: Use GROUP BY EmployeeID, then filter groups with HAVING instead of WHERE.

Expected Output



EmployeeID	OrderCount
4	156
3	127
1	123
8	104
2	96
7	72
6	67

#	Time	Action
1	17:29:56	SELECT EmployeeID, COUNT(OrderID) AS OrderCount FROM Orders G...

.

Question 6

Task: Write a query that lists each shipper (ShipVia) and counts the number of orders assigned to them. Label the count OrderCount. Order results by ShipperID ascending.

Hint: Use GROUP BY on the ShipVia column in the Orders table.

Expected Output

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

ShipperID	OrderCount
1	249
2	326
3	255

Result 4

Output

Action Output

#	Time	Action	Message
1	17:32:14	SELECT ShipVia AS ShipperID, COUNT(OrderID) AS OrderCount F...	3 row(s) returned

---

Question 7

Task: List every product together with the name of its category. Display ProductID, ProductName, and CategoryName. Order alphabetically by CategoryName, then by ProductName within each category.

Hint: Join the Products table to the Categories table on the CategoryID column.

Expected Output

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

ProductID	ProductName	CategoryName
1	Chai	Beverages
2	Chang	Beverages
39	Chartreuse verte	Beverages
38	Cte de Blaye	Beverages
24	Guaran Fantstica	Beverages

Result 5

Output

Action Output

#	Time	Action	Message
1	17:34:44	SELECT p.ProductID, p.ProductName, c.CategoryName FROM Products...	77 row(s) returned

---

Question 8

Task: Write a query that shows each order and the company name of the customer who placed it. Display OrderID, OrderDate, and CompanyName. Order by OrderDate from most recent to oldest.

Hint: Join the Orders table to the Customers table on CustomerID.

Expected Output

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

OrderID	OrderDate	CompanyName
11076	1998-05-06 00:00:00	Bon app'
11077	1998-05-06 00:00:00	Rattlesnake Canyon Grocery
11075	1998-05-06 00:00:00	Richter Supermarkt
11074	1998-05-06 00:00:00	Simons bistro
11072	1998-05-05 00:00:00	Ernst Handel

Result 6

Output

Action Output

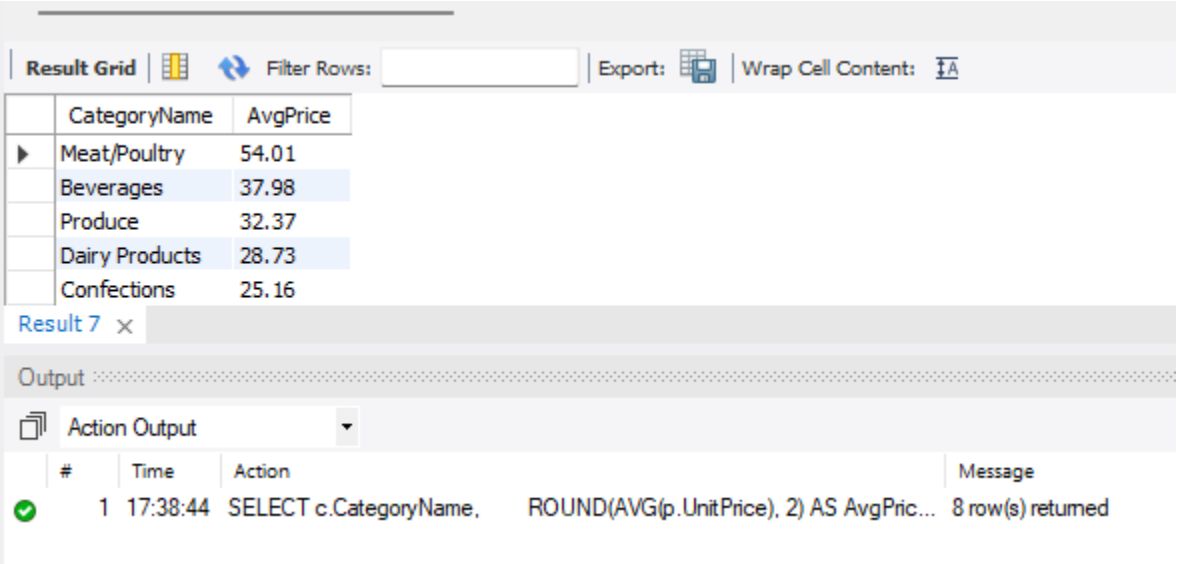
#	Time	Action	Message
1	17:36:48	SELECT o.OrderID, o.OrderDate, c.CompanyName FROM Orders AS o J...	830 row(s) returned

Question 9

Task: Write a query that calculates the average unit price of products in each category. Display CategoryName and the average price labeled AvgPrice, rounded to 2 decimal places. Only show categories where the average price is greater than $20. Order by AvgPrice descending.

Hint: Join Products and Categories, use AVG() with GROUP BY CategoryID, filter with HAVING, and use ROUND().

Expected Output



CategoryName	AvgPrice
Meat/Poultry	54.01
Beverages	37.98
Produce	32.37
Dairy Products	28.73
Confections	25.16

#	Time	Action	Message
1	17:38:44	SELECT c.CategoryName, ROUND(AVG(p.UnitPrice), 2) AS AvgPric...	8 row(s) returned

Question 10

Task: Write a query that lists each employee's full name along with the total number of orders they have processed. Show FullName (combined FirstName + LastName) and OrderCount. Only include employees who have processed at least 70 orders. Order by OrderCount descending, then by FullName ascending.

Hint: Join Employees and Orders on EmployeeID, aggregate with COUNT and GROUP BY, then filter with HAVING.

Expected Output

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

EmployeeID	FullName	OrderCount
4	0	156
3	0	127
1	0	123
8	0	104
2	0	96

Result 8

Output

Action Output

#	Time	Action	Message
1	17:40:33	SELECT e.EmployeeID, e.FirstName + ' ' + e.LastName AS FullName...	6 row(s) returned

Part B Solutions

Question 1

```sql
-- Question 1
SELECT ProductID, ProductName, UnitPrice, UnitsInStock
FROM Products
ORDER BY UnitPrice DESC;
```

Explanation

- ORDER BY controls the sequence of rows in the result set.
- DESC (descending) sorts from the largest value to the smallest — here, most expensive products appear first.
- ASC (ascending) is the default and does not need to be written explicitly.
- ORDER BY is placed after the WHERE clause (if any) and is the last clause in a basic SELECT statement.

Question 2

```sql
-- Question 2
SELECT CustomerID, CompanyName, ContactName, Country
FROM Customers
ORDER BY Country ASC, CompanyName ASC;
```

Explanation

- Multiple columns in ORDER BY create a multi-level sort.
- The first column listed (Country) is the primary sort key.
- The second column (CompanyName) is the secondary sort key — it only comes into play when two or more rows share the same Country value.
- ASC is the default direction but is written here for clarity.

Question 3

```sql
-- Question 3
SELECT CategoryID, COUNT(ProductID) AS ProductCount
FROM Products
GROUP BY CategoryID
ORDER BY ProductCount DESC;
```

Explanation

• GROUP BY CategoryID splits the products into groups, one per category.

• COUNT(ProductID) then counts how many products exist within each group.

• ORDER BY ProductCount DESC sorts by the alias, so the largest group appears first.

• This is a classic aggregation pattern: SELECT a grouping column + aggregate, GROUP BY that column, then ORDER BY the result.

Question 4

```sql
-- Question 4
SELECT OrderID,
       ROUND(SUM(UnitPrice * Quantity * (1 - Discount)), 2) AS Revenue
FROM `Order Details`
GROUP BY OrderID
ORDER BY Revenue DESC;
```

Explanation

• SUM() is an aggregate that totals values across all rows in each group.

• The expression UnitPrice * Quantity * (1 - Discount) computes the discounted line total for each row.

• GROUP BY OrderID groups all line items belonging to the same order so SUM adds them together.

• The table name is [Order Details] — square brackets are required in SQL Server when a table name contains a space.

Question 5

```sql
-- Question 5
SELECT EmployeeID, COUNT(OrderID) AS OrderCount
FROM Orders
GROUP BY EmployeeID
HAVING COUNT(OrderID) > 50
ORDER BY OrderCount DESC;
```

Explanation

• HAVING is to GROUP BY what WHERE is to individual rows — it filters after aggregation.

• You cannot use WHERE to filter by an aggregate (e.g., WHERE COUNT(...) > 50) because WHERE runs before grouping.

• HAVING COUNT(OrderID) > 50 keeps only groups (employees) where the order count exceeds 50.

• Always remember: WHERE → GROUP BY → HAVING → ORDER BY is the logical execution order

Question 6

```sql
-- Question 6
SELECT ShipVia AS ShipperID,
       COUNT(OrderID) AS OrderCount
FROM Orders
GROUP BY ShipVia
ORDER BY ShipVia ASC;
```

Explanation

• GROUP BY ShipVia groups all orders by the shipper used.

• COUNT(OrderID) counts the orders in each group.

• AS ShipperID gives the grouped column a more descriptive alias.

• This pattern is useful for quickly discovering the distribution of orders across shipping carriers.

Question 7

```sql
-- Question 7
SELECT p.ProductID, p.ProductName, c.CategoryName
FROM Products AS p
JOIN Categories AS c ON p.CategoryID = c.CategoryID
ORDER BY c.CategoryName ASC, p.ProductName ASC;
```

Explanation

• JOIN (INNER JOIN) combines rows from two tables where the join condition is met.

• ON p.CategoryID = c.CategoryID matches each product to its category using the shared key.

• Table aliases (p, c) make the query shorter and clarify which table each column comes from — especially important when both tables share column names.

• The ORDER BY clause uses two sort keys: primary by CategoryName, secondary by ProductName.

Question 8

```sql
-- Question 8
SELECT o.OrderID, o.OrderDate, c.CompanyName
FROM Orders AS o
JOIN Customers AS c ON o.CustomerID = c.CustomerID
ORDER BY o.OrderDate DESC;
```

Explanation

• The JOIN links orders to customers through the shared CustomerID foreign key.

• Without the JOIN you could only see the CustomerID code in Orders; with it you see the human-readable company name from Customers.

• ORDER BY o.OrderDate DESC shows the newest orders at the top — a common reporting requirement.

• Qualifying column names with table aliases (o.OrderID, c.CompanyName) prevents ambiguity.

Question 9

```sql
-- Question 9
SELECT c.CategoryName,
       ROUND(AVG(p.UnitPrice), 2) AS AvgPrice
FROM Products AS p
JOIN Categories AS c ON p.CategoryID = c.CategoryID
GROUP BY c.CategoryID, c.CategoryName
HAVING AVG(p.UnitPrice) > 20
ORDER BY AvgPrice DESC;
```

Explanation

• AVG() calculates the mean of UnitPrice within each category group.

• ROUND(..., 2) trims the result to 2 decimal places for a clean currency display.

• GROUP BY must include c.CategoryName (in addition to c.CategoryID) because it appears in the SELECT — a requirement of standard SQL.

• HAVING AVG(p.UnitPrice) > 20 filters out categories with a lower average price — note the aggregate is repeated in HAVING, not the alias.

Question 10

```sql
-- Question 10
SELECT e.EmployeeID,
       e.FirstName + ' ' + e.LastName AS FullName,
       COUNT(o.OrderID) AS OrderCount
FROM Employees AS e
JOIN Orders AS o ON e.EmployeeID = o.EmployeeID
GROUP BY e.EmployeeID, e.FirstName, e.LastName
HAVING COUNT(o.OrderID) >= 70
ORDER BY OrderCount DESC, FullName ASC;
```

Explanation

- The JOIN links each order to the employee who handled it via EmployeeID.
- String concatenation (FirstName + ' ' + LastName) builds the full name directly in the SELECT.
- GROUP BY must list all non-aggregated columns in SELECT: EmployeeID, FirstName, and LastName.
- HAVING COUNT(...) >= 70 keeps only high-volume employees.
- The two-column ORDER BY demonstrates how to resolve ties — here, alphabetically by full name when two employees have the same order count.